U of T Course Compass

Shayan Bhatti, Jacob Chislett, Ethan Diep, Shuhan Yuan

April 2, 2026

Problem Description and Research Question

The University of Toronto provides a detailed academic calendar informing students about all available courses, prerequisites, and program requirements. While this information is publicly available, it is displayed in a way that is overwhelming and entirely text based, making it difficult to understand. For example, consider the prerequisites for **MAT244H1 – Introduction to Ordinary Differential Equations**:

> (MAT133Y1/ MAT135H1/ MAT135H5/ MATA35H3/ MATA30H3/ MATA31H3,
> MAT136H1/ MAT136H5/ MATA36H3/ MATA37H3)/ MAT135Y5/ MAT137Y1/ MAT137Y5/
> (MAT137H5, MAT139H5)/ MAT157Y1/ MAT157Y5/ (MAT157H5, MAT159H5),
> MAT223H1/ MATA23H3/ MAT223H5/ MAT240H1/ MAT240H5

When reading this, a student must first understand what the slashes, commas, and parentheses mean. If a student clicks on any single course from the list to see its details, it will lead to a similarly overwhelming list, forcing the student to repeatedly look up course codes, interpret notation, and mentally map all the multi-level connections between courses. This makes it extremely difficult to visualize the full set of courses required to reach a target course, assess progression, or plan an efficient academic path. Beyond the structural confusion of prerequisites, the calendar also provides no information on course quality, workload, or student satisfaction, leaving students without the guidance on which courses to prioritize, making it difficult to make informed decisions about their program progression. To address this, there is a clear need for a tool that can organize and visualize course prerequisites in a way that is intuitive and interactive. Our project, U of T Course Compass, aims to address this need: **How can we make the course selection process more accessible and understandable for students at U of T?**

1 Datasets:

Our project uses two primary datasets: data scraped from the University of Toronto's Academic Calendar and course evaluation data from U of T's course evaluations portal.

Dataset 1: Academic Calendar:
The academic calendar dataset was collected by scaping the UofT Arts and Science Academic Calendar website (https://artsci.calendar.utoronto.ca) using requests library and BeautifulSoup. For each course, we extracted the course code, full course name, description, and prerequisite string. We did not extract data for all the courses, only those from the Mathematics, Computer Science, Physics, Economics, and Statistical Sciences departments.

After our dataset is gathered, it is not in a format that can easily have computations performed on it. We must first clean and parse the data, and this is all handled by the `PrerequisiteTreeLoader` class in the `academic_calendar_reader` file. the `prerequisite_tree_loader` has a method `load_from_programs` which initializes all the prerequisite trees for every course in the given programs by calling the other helper methods in the file. This involves accessing the raw prerequisite strings from the academic calendar website, stepping through the strings character by character and identifying connectives ('/', ',') and course codes, and identifying the underlying logical structure of the prerequisite strings. Once all the courses have been loaded, `PrerequisiteTreeLoader` has a `save_to_file` method which saves all information gathered from the website and parsed into a json file. This includes the course codes, names, descriptions, and properly formatted prerequisite strings. This is to make sure that the long data gathering phase

does not need to be run every time the application is run, so that on subsequent runs all the information can simply be loaded from the json file. We consider that json file to be the properly cleaned and formatted dataset.

The prerequisite string has specific notation that is defined by the calendar: slashes (/) separate alternative prerequisites, commas (`,`) separate cumulative requirements, and parentheses creates a group of courses. Some courses also specify minimum grade requirements alongside their prerequisite course codes (e.g., `CSC148H1(60%)`). However, these minimum grade requirements are not standardized across departments or even within departments, so extracting this information is not trivial. Keeping these in mind, our program parses these strings into a structured internal format (`CourseTree`) that is in `course_tree.py`. Each node of the tree is either a course code (a leaf node), or a logical operator (`ALL` or `CHOOSE`) grouping its children. Grade requirements are stored as integer attributes on leaf nodes (`-1` if no grade requirement is specified).

Links to individual course and program pages on the Academic Calendar are generated using helper functions in `link_of_course.py`, which format course and program codes into their corresponding URLs for display to users.

Dataset 2: UofT Course Evaluations:
The course evaluation dataset was collected by scraping UofT's course evaluations website using `Selenium`, a browser automation library, as implemented in `course_evals_parser.py`.

We used `By`, `webdriver`, and `WebDriverWait` from `Selenium` to control a Chrome browser. We saved the link to UofT's course evaluations website in a variable and used `parse_department` to scrape all course evaluation data under the departments of Computer Science, Statistical Sciences, Mathematics, Economics, and Physics. Specifically, it enters these department names into the search bar and selects the first result from the drop-down menu.

Next, we used `parse_table` and `_parse_page` to paginate through the results table row by row. Each row of the table was extracted and appended to a `.csv` output file.

Each record in the resulting CSV contains the following fields: Division, Department, Course name and code, Term, Year, five general evaluation items (rated 1–5), Instructor Enthusiasm, Course Workload, Recommend Course rating, Number of students invited, and Number of responses. Note that a sample of the scraped output is available in `demo.csv`, which contains data from the first two pages of the evaluations table. After creating it as a csv, we used the column 3, 6, 8-16 of the csv data as they were the only ones we needed.

2 Computational Overview:

2.1 Use of Trees to Model Data:

Our project uses two kinds of data to represent the UofT course domain: academic calendar data and course evaluation data. The academic calendar data is loaded using `load_course_information()` in `academic_calendar_reader.py`, which returns a dictionary mapping each course code (a string) to its prerequisite string, and another dictionary that maps each course code to its full name and description. In addition there are the course evaluation data which are stored in `.csv` files such as `mathematics.csv` and `demo.csv`, scraped using the `CourseEvalsParser` class in `course_evals_parser.py`. These are quantitative data that are later used for the computations (more on it below)

Trees play the central role in our data representation. The `CourseTree` class defined in `course_tree.py` is the core data structure of the entire project. Each `CourseTree` node has three attributes: `_root`, which stores either a course code, `'ALL'`, or `'CHOOSE'`; `_required_grade`, an integer storing the minimum grade requirement for that course (-1 if none); and `_subtrees`, a list of child `CourseTree` nodes. Leaf nodes always represent individual course codes, while internal nodes represent either logical relationships (`ALL` meaning every child course is required, and `CHOOSE` meaning at least one child is required) or more course codes. This maps directly onto the academic calendar's notation where commas indicate cumulative requirements and slashes indicate alternatives. The tree is hierarchical by design, allowing the user to trace a clear path of all courses needed to reach a target course, at any level of depth.

For the `CourseTree` class that we use to store prerequisite information for a given course, we borrowed several methods from the in-class implementation, and also added several of our own. All of the basic features like getting a string representation of the tree were taken, but we added several computations like generating a course tree from a prerequisite string, and simplifying a course tree based on courses that were taken by the user.

2.2 Major Computations

Building the Prerequisite Tree

The prerequisite tree is built using the static method `CourseTree.generate_course_tree()` in `course_tree.py`. This method works recursively by examining the outermost logical structure of a prerequisite string. It uses two helper functions from `string_methods.py`: `has_unnested()`, which checks whether a given character appears outside of any parentheses, and `unnested_split()`, which splits the string at all unnested instances of a given character. If the string contains unnested commas, an `ALL` node is created with each comma-separated component as a subtree. If it contains unnested slashes, a `CHOOSE` node is created instead. If the string is a single graded course code (validated by `is_graded_course_code()` in `string_methods.py`), a leaf node is created with the course code and grade requirement stored directly on the node. If the string is wrapped in redundant parentheses, the outer layer is stripped and the function recurses on the inner content. This continues until all components have been reduced to leaf nodes. However, this will only show direct prerequisites to a given course. When the `AcademicCalendarReader` is being generated in the `load_from_file` method, the program loops through every course tree and replaces the leaf nodes with references to the courses they correspond to. This makes all of the course trees interconnected in memory so that each course then shows the full list of prerequisites, rather than just the first layer.

Building the Post-requisite Tree

The post-requisite tree is generated in `academic_calendar_reader.py`. The method `get_postrequisite_tree` of `PrerequisiteTreeLoader` loops through every course that has a corresponding prerequisite tree, and checks whether the provided course is a direct prerequisite to any of the courses being looped through. If so, then that course is added as a subtree to the `postrequisite_tree` which is finally returned.

Simplifying the Tree Based on Taken Courses

Tree simplification is handled by the `simplify_tree()` method in `CourseTree`. It takes a dictionary mapping course codes to the grades the user achieved, and recursively traverses the tree. For nodes representing individual courses, it checks whether the course appears in the dictionary with a sufficiently high grade using `_course_completed()`. If so, the node is replaced with an empty tree. For `ALL` nodes, if all subtrees become empty the node collapses entirely; otherwise empty subtrees are pruned using `_remove_empty_subtrees()`. For `CHOOSE` nodes, if any one subtree becomes empty the entire node collapses, since one of the alternatives has been satisfied. If a node ends up with only one subtree after pruning, that extra layer is removed to keep the tree clean.

Consolidate Rating:
Within `consolidate_rating.py`, the `group_by_course_code` function takes in the CSV, groups the course evaluation data by course code, and outputs it to a JSON file. Then, the `compute` function takes in the raw JSON file and performs the following computations:

1. Calculates the total number of student responses, the number of times the course was offered, and the number of years it was offered.
2. Extracts a history of all terms the course was offered and filters for just the summer terms.
3. Takes the values for specific variables across the years, calculates their historical averages, and combines them into broader evaluation categories.

Within the course_evals_parser.py there is a demo. It showcases the course evaluation scraper by automatically scraping the first two pages of UofT's course evaluations table and saving the results to `demo.csv`
To run the demo: run the _main_ block, A Chrome browser window will open automatically and navigate to UofT's course evaluations portal. The scraper will paginate through the first two pages of the table, printing each scraped row to the console. and then close the window automatically. (Note that this can only be ran on Chrome)
From this computation, we were able to implement the feature that informs the students whether the course is offered in the summer

2.3 Visual and Interactive Reporting

One of the main computations our program performs is recursively drawing a course tree of any size to the screen. The main visualization logic is implemented in the `draw_tree_visualization` method in the Tree class and the python library `pygame` was central to the implementation. The `draw_tree_visualization` method recursively traverses the `CourseTree` to determine the position of each node and its children. At each recursive step, the method calculates the total horizontal space required by all child nodes using the `tree_width` method, which sums the widths of all subtrees at the current level. This total width is then used to determine the starting horizontal position (`start_x_pos`) for the leftmost child so that the children are centered beneath the parent node. Each child subtree is allocated a horizontal space proportional to its width, and its center is positioned in the middle of that space. Before recursively drawing each child, the method draws a line connecting the parent node to the child node. This line is drawn from the bottom center of the parent rectangle to the top center of the child rectangle using `pygame.draw.line`. The `draw_node` method is responsible for rendering each individual node in detail. It creates a `pygame.Rect` to represent the node.

2.4 New Python Libraries

Our project makes use of three new Python libraries: `BeautifulSoup`, `Selenium`, and `pygame`, `textwrap`, `webbrowser`

`BeautifulSoup` from the `bs4` library was used to scrape the UofT Academic Calendar in `academic_calendar_reader.py`. It was initialized with `BeautifulSoup(webpage.text, features="html.parser")` to convert raw HTML retrieved by `requests.get()` into a structured, searchable parse tree. Methods such as `soup.find()` and `soup.find_all()` were then used to locate specific HTML elements containing course codes, course names, descriptions, and prerequisite strings, with the `.text` attribute used to extract the plain text content from those elements.

`Selenium` was used in `course_evals_parser.py` to scrape the UofT course evaluations portal. A `webdriver` was used to create an instance of the Chrome browser and open it. Below is a summary of the imported features.

- `WebDriverWait` provides a timeout feature. Since we need to check whether the HTML elements we are looking for are visible on screen, we must wait for some duration of time. `WebDriverWait` sets a max duration to wait before timing out and throwing an error.
- `WebElement` is used to represent a DOM element, which can be interacted with (e.g. by clicking). Text can also be extracted from it. This is how we get the table data.
- `By`: To select a specific DOM element, the Selenium `By` class is used. `By` allows us to specify our reference to the desired element.
- `WebElement` represents a single HTML element on a web page. It provides methods to interact with DOM elements programmatically — including clicking, typing, reading attributes, and checking visibility or state. Elements are located via the driver's `find_element()` method using a `By` locator strategy, such as `By.ID`, to target specific elements, and can then be acted upon directly through the returned `WebElement` object.
- `expected_conditions` was used in our custom `WebElementHelper` class made to compartmentalize the code required for finding a specified Selenium `WebElement` on the page. `expected_conditions` provides two useful methods: `staleness_of` and `visibility_of_element_located`.
  - `staleness_of` is used to check whether a `WebElement` is stale. A `WebElement` refers to an HTML element on the DOM, so once it has been removed (like when the course evaluations table is removed and then added again during pagination before rendering), the reference is stale. `staleness_of` allows us to check if a `WebElement` is stale to update it.
  - `visibility_of_element_located` checks whether an element has been loaded onto the website and is in view.

`Textwrap` library was used to format text across multiple lines, as some content exceeded the width of the display area and required line wrapping.

The `webbrowser` library was used to open links in a browser if the user clicked read more on the info box

By using these libraries, we were able to parse through the course website and get the necessary data. After computing and cleaning the data, we got information that is later showed to the user. For example: course rating,

profs_by_rating, and historical_offerings...

`pygame` was used to build the entire interactive visualization. Specifically, `pygame.display.set_mode()` was used to initialize the application window, `pygame.draw.rect()` and `pygame.draw.line()` were used to render course nodes and the edges connecting them, and `pygame.font.Font().render()` was used to draw course code labels onto each node. User input was handled through pygame's event system, detecting keyboard events for WASD navigation and mouse click events for selecting individual course nodes to view additional information.

3 Program Instructions:

Getting Started To load the dataset from the zip file that we uploaded to markus, just extract all the contents to the root directory of the project.

- Open the `main.py` file and run the program.
- The program will begin with a Start Screen. Press **ENTER** to proceed.

Course Selection Screen
You are now in the Course Selection Screen. This is where you can save courses in the program.
To add a course:

- Click on the **Course Code** field and enter the course code.
- Click on the **Grade** field and enter the grade.
- Click **Add Course** to save the course.

Any courses saved will allow the program to perform computations such as simplifying the course tree or calculating the optimal path. Now, you can press **ENTER** to go to the Main Screen for data visualizations.
Main Screen Controls

1. **Search for a Course**
   - Use the **Search Course** bar at the top left to enter a course code or name.
   - This sets the course for all other features: Course Tree, Optimal Path, Course Comparison, and Summer Offering.
2. **Course Tree**
   - Choose the type of tree: Prerequisite or Postrequisite.
   - You can also enable **Simplify Pre-req Tree** to display only courses you haven't completed.
   - Click **Generate** to visualize the course tree in the right panel.
3. **Optimal Path**
   - Select one of the following optimization factors:
     - **Overall Satisfaction:** Focus on courses with higher student satisfaction.
     - **Workload:** Minimize workload along the path.
     - **Cognitive Growth:** Focus on courses that maximize learning growth.
   - Click **Generate** to display the recommended path based on the courses you have taken.
4. **Summer Offering**
   - Check if the selected course has historically been offered during the summer.
   - Click **Check** to display the result in the right panel.
5. **Course Difference**
   - Enter a second course in the "Course to Compare" field.
   - Click **Generate** to display a chart between the two courses.

4 Changes to Our Project Plan:

After much discussion amongst ourselves, we planned to remove the "Unintended minor" mentioned in the proposal. We feel that this feature isn't too helpful realistically speaking; the user could always notice any potential minor they could pursue from the program requirements. Program requirements are also significantly less straightforward than course prerequisites, so each program would have to be handled separately that was infeasible given the scope of the project. Hence, that plan was removed.

Additionally, we also mutated the feature "Bottleneck Courses" mentioned in the proposal to another feature called the "course post-requisite difference". Essentially, it shows all the additional courses one could take if one takes a course A as opposed to course B. Vice Versa, if course B was taken instead of course A. For example, Consider the direct postrequisites of MAT137Y1 and MAT157Y1: this feature would show many courses that are exclusive to MAT157Y1 (like MAT257Y1, MAT267H1, MAT327H1) that taking only MAT137Y1 is not adequate preparation for. The purpose of the feature is to allow students to make more informed decision, in this case, whether to take MAT137/157 after knowing which courses each option would provide.

Another feature we removed is the "Course heat map" mentioned in the proposal. We found it redundant because it did not provide the user with enough useful information, and they could always find the course difficulty in the pop up menu. Therefore, we didn't think it would make that much of impact.

Another feature we added it called "Summer Course Offerings". This feature would provide the user the amount of times the course is offered in summer from the information given in the Course Evaluations. This feature is implemented to help the students have a informed guess whether the course would be offered for the coming summer semester.

5 Discussion section:

Our program successfully fulfills the goal of making the course selection process more accessible and understandable for UofT students. By automatically generating prerequisite and post-requisite trees from the academic calendar and displaying them interactively through pygame, students are able to visualize complex multi-level course relationships that would otherwise require extensive manual navigation of the calendar's text-based notation. The tree simplification feature further personalizes this by allowing students to input their completed courses and instantly see only what remains, and the course evaluation information provides an additional layer of information about workload and satisfaction that the academic calendar does not offer at all.

One obstacle we encountered was the unorganized text found in the Academic calendar. It was difficult to parse through the string of text, and find the necessary course with inconsistent text patterns. For example, finding the correct grade prerequisite was difficult because of the lack of standardization across course prerequisites. Some examples of how this is presented are: CSC110Y1(70% or higher)", "60% or higher in CSC148H1", or "75% or greater in CSC338H5". We found it hard to parse through these different string, and save the valid course code, and the mark required for that course. Also, since our application is intended for students at the St. George campus, we had to eliminate courses from other campuses while preserving the logical structure of the prerequisites. This required nontrivial parsing of the prerequisite string and careful consideration of the way courses could be combined with each other.

A second limitation of our program came from the use of Pygame for the user interface. While Pygame is well-suited for game development, it is not the best for creating complex or feature rich UI components. For example, implementing common UI elements such as scroll bars was nontrivial and time-consuming, and thus were not included in the current version. Even simpler tasks, such as displaying multi-line text, required the use of Python's textwrap module and lengthy custom functions. These challenges limited our ability to create a more polished and flexible interface.
Additionally, as the number of nodes increased in larger course trees, the program experienced performance slowdowns due to having to render numerous visual elements. This further highlighted the need for UI optimization and more efficient rendering strategies. In the future, we aim to explore alternative UI frameworks better suited for complex applications, such as Tkinter, PyQt, or Kivy, which would allow richer interactive elements and greater control over layout and styling. Additionally, we can implement rendering optimizations for large course trees, such as drawing only visible nodes or using more efficient graphics libraries, to improve performance and responsiveness.

Lastly, we can enhance the UI with additional interactive features, including scrollable panels, zooming, and collapsible tree nodes, to improve usability for extensive course catalogs.

For further exploration, the most natural next step would be expanding data collection to cover all departments offered at UofT, making the tool useful for the entire student population rather than a subset. Further improvements to the pygame interface, such as better layout algorithms for very large trees, would also improve the user experience. Overall, our program demonstrates that trees are a natural and effective structure for representing course prerequisite data, and that combining scraped academic data with an interactive visualization can meaningfully improve how students plan their academic paths at UofT.

References:

Breuss, M. (n.d.). *Beautiful Soup: Build a web scraper with Python – Real Python*. Real Python. Retrieved March 4, 2026, from https://realpython.com/beautiful-soup-web-scraper-python/

Ellis, M., & Quarshie, M. (2025, February 3). Student course evaluations rank statistics department lowest, small humanities centres highest. *The Varsity*. Retrieved March 4, 2026, from https://thevarsity.ca/2025/02/03/student-course-evaluations-rank-statistics-department-lowest-small–humanities-centres-highest/

Pygame front page — pygame v2.6.0 documentation. (n.d.). Pygame. Retrieved March 4, 2026, from https://www.pygame.org/docs/

University of Toronto. (n.d.). *2025–26 academic calendar — Academic calendar*. Retrieved March 4, 2026, from https://artsci.calendar.utoronto.ca/

University of Toronto. (n.d.). *U of T course evaluations – Faculty of Arts & Science UG*. Course Evals. https://course-evals.utoronto.ca/BPI/fbview.aspx?blockid=OjzZ9-LrM-peMm6q2u&userid=cYQTzF-3fo2ufLLGH26rS-YRliCjyxiGeI8T&lng=en